

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

45 - Parallel Processors

Introduction

- Create powerful computers by connecting many existing smaller ones: multiprocessors
- Multiprocessor (MP) software must be designed to work with a variable number of processors
- Energy has become overriding issue for both MPs and datacenters
- Replacing large inefficient processors with many smaller, efficient processors can deliver better performance per joule

Goal

- Connecting multiple computers to get higher performance
 - Multiprocessors
 - Scalability, availability, power efficiency
- Task-level (process-level) parallelism
 - High throughput for independent jobs
- Parallel processing program
 - Single program run on multiple processors
- Multicore microprocessors
 - Chips with multiple processors (cores)

Multiprocessor

- A computer system with at least two processors
- In contrast to a uniprocessor, which has one

Multiprocessor

- SW must be designed to work with a variable number of processors
- Power has become overriding issue for both datacenters and microprocessors
- Replacing large inefficient processors with many smaller, efficient processors can deliver better performance per watt or per joule, if SW can efficiently use them

Availability

- Some designs support operation in the presence of broken HW
- If a single processor fails in a multiprocessor with n processors, the system would continue to provide service with $n - 1$ processors
- Multiprocessors improve availability

High Performance

- Can mean high throughput for independent jobs, called **job-level parallelism** or **process-level parallelism**
- These parallel jobs are independent applications, and they are an important and popular use of parallel computers
- This approach is in contrast to running a single job on multiple processors
- Use term **parallel processing program (PPP)** to refer to single program that runs on multiple processors simultaneously

Cluster

- Set of computers connected over a LAN that functions as a single large multiprocessor
- Scientific problems that have needed much faster computers
 - Some of these problems can be handled simply, using a **cluster** composed of microprocessors housed in many independent servers or PCs
- In addition, clusters can serve demanding applications, such as search engines, Web servers, email servers, and databases

Multicore

- A microprocessor containing multiple processors (“cores”) in a single integrated circuit
- Power problem
 - More processors per chip rather than higher CRs and improved CPI
- Called **multicore microprocessors**
- Processors are often called **cores** in a multicore chip

Multicore

- Shared Memory Processors (SMPs), share single physical address space
- Number of cores is expected to double every two years
- Programmers who care about performance must become parallel programmers
- Sequential code now means slow code

Hardware and Software

- Hardware
 - Serial: e.g., Pentium 4
 - Parallel: e.g., quad-core Xeon e5345
- Software
 - Sequential: e.g., matrix multiplication
 - Concurrent: e.g., operating system
- Sequential/concurrent SW can run on serial/parallel HW
 - Challenge: making effective use of parallel hardware

Hardware and Software

		Software	
		Sequential	Concurrent
Hardware	Serial	Matrix Multiply written in MatLab running on an Intel Pentium 4	Windows Vista Operating System running on an Intel Pentium 4
	Parallel	Matrix Multiply written in MATLAB running on an Intel Core i7	Windows Vista Operating System running on an Intel Core i7

Difficulty of Creating PPPs

- Uniprocessor design techniques such as superscalar and out-of-order execution take advantage of ILP, without programmer involvement
- Such innovations reduced demand for rewriting programs for MPs
 - Since programmers could do nothing and yet their sequential programs would run faster on new computers

Parallel Programming

- Parallel SW is the problem
- Need to get significant performance improvement
 - Difficult to write SW that uses multiple processors to complete one task faster, and the problem gets worse as the number of processors increases
 - Otherwise, just use a faster uniprocessor, since it's easier!
- Difficulties
 - Partitioning
 - Coordination
 - Communications overhead

Partitioning

- Analogy of 8 reporters trying to write a single story in hopes of doing the work 8 times faster
- To succeed, the task must be broken into 8 equal-sized pieces, because otherwise some reporters would be idle while waiting for the ones with larger pieces to finish

Coordination (Synchronization)

- As processors operating in parallel will normally share data, they also need to coordinate when operating on shared data
- Otherwise, one processor could start working on data before another is finished with it

Communication Overhead

- Reporters would spend too much time communicating with each other instead of writing
- Parallel programming: challenges include scheduling, load balancing, time for synchronization, and overhead for communication between parties
- Challenge is stiffer with the more reporters for a newspaper story and the more processors for parallel programming

SISD, MIMD, SIMD, SPMD

- **SISD** Single Instruction stream, Single Data stream
 - Uniprocessor
- **MIMD** Multiple Instruction streams, Multiple Data streams
 - Multiprocessor
- **SPMD** Single Program, Multiple Data streams
 - Conventional MIMD programming model, where a single program runs across all processors
- **SIMD** or Single Instruction stream, Multiple Data streams
 - Same instruction is applied to many data streams, as in a vector processor

GPU

- The original justification for adding SIMD instructions to existing architectures was that many microprocessors were connected to graphics displays in PCs and workstations, so an increasing fraction of processing time was used for graphics.
- As **Moore's Law** increased the number of transistors available to microprocessors, it therefore made sense to improve graphics processing

GPU

- Major driving force for improving graphics processing was the computer game industry, both on PCs and in dedicated game consoles such as the Sony PlayStation
- Rapidly growing game market encouraged many companies to make increasing investments in developing faster graphics HW
- This positive feedback loop led graphics processing to improve at a faster rate than general-purpose processing in mainstream microprocessors

GPU

- Given that the graphics and game community had different goals than the microprocessor development community, it evolved its own style of processing and terminology
- As the graphics processors increased in power, they earned the name *Graphics Processing Units* or *GPUs* to distinguish themselves from CPUs

History of GPUs

- Early video cards
 - Frame buffer memory with address generation for video output
- 3D graphics processing
 - Originally high-end computers (e.g., SGI)
 - Moore's Law $\Rightarrow$ lower cost, higher density
 - 3D graphics cards for PCs and game consoles
- Graphics Processing Units
 - Processors oriented to 3D graphics tasks
 - Vertex/pixel processing, shading, texture mapping, rasterization

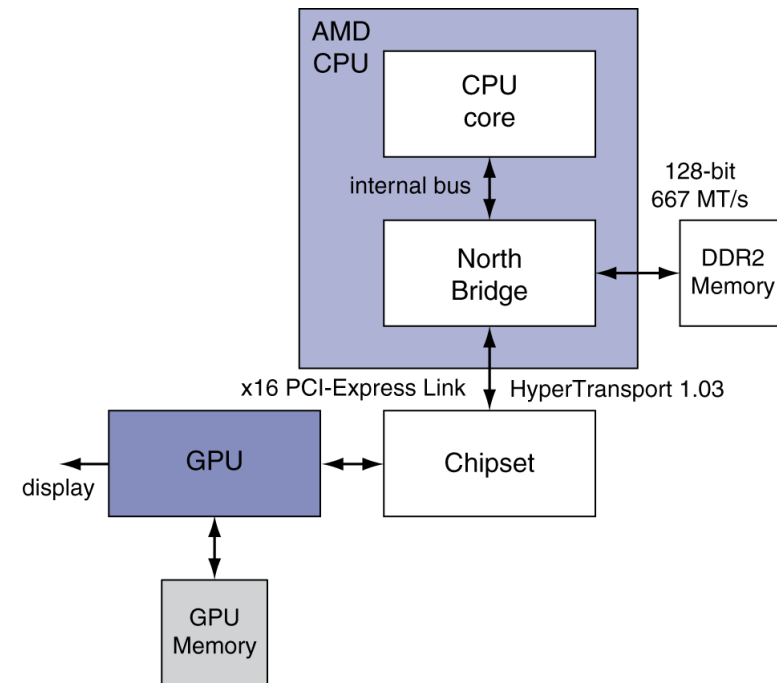
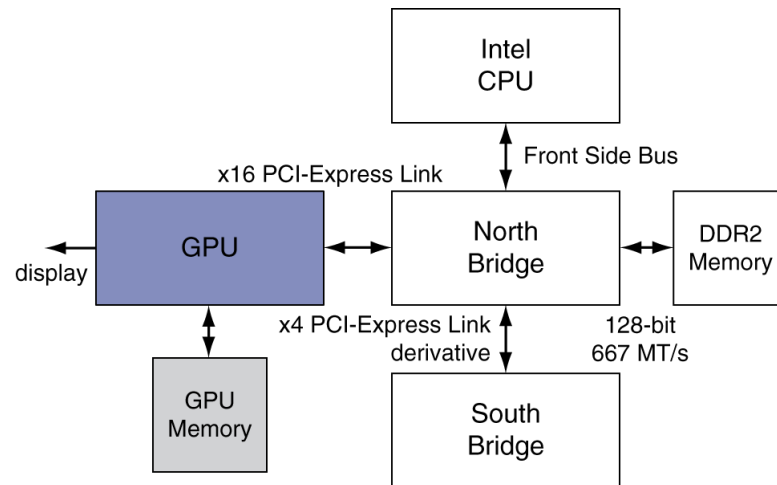
GPU

- For a few hundred dollars, anyone can buy a GPU today with hundreds of parallel floating-point units, which makes high-performance computing more accessible
- Interest in GPU computing blossomed when this potential was combined with a programming language that made GPUs easier to program
- Hence, many programmers of scientific and multimedia applications today are pondering whether to use GPUs or CPUs

GPUs vs CPUs

- GPUs are accelerators that supplement a CPU, so they do not need be able to perform all the tasks of a CPU
- Dedicate all their resources to graphics
- GPU problems sizes are typically hundreds of megabytes to gigabytes, but not hundreds of gigabytes to terabytes

Graphics in the System



GPU Programming

- Although GPUs were designed for a narrower set of applications, some programmers wondered if they could specify their applications in a form that would let them tap the high potential performance of GPUs
- Developed C-inspired programming languages to allow them to write programs directly for GPUs
 - Example NVIDIA's CUDA (Compute Unified Device Architecture), which enables the programmer to write C programs to execute on GPUs, albeit with some restrictions

Putting GPUs into Perspective

Feature	Multicore with SIMD	GPU
SIMD processors	4 to 8	8 to 16
SIMD lanes/processor	2 to 4	8 to 16
Multithreading hardware support for SIMD threads	2 to 4	16 to 32
Typical ratio of single precision to double-precision performance	2:1	2:1
Largest cache size	8 MB	0.75 MB
Size of memory address	64-bit	64-bit
Size of main memory	8 GB to 256 GB	4 GB to 6 GB
Memory protection at level of page	Yes	Yes
Demand paging	Yes	No
Integrated scalar processor/SIMD processor	Yes	No
Cache coherent	Yes	No

GPU Architectures

- Processing is highly data-parallel
 - GPUs are highly multithreaded
 - Use thread switching to hide memory latency
 - Less reliance on multi-level caches
 - Graphics memory is wide and high-bandwidth
- Trend toward general purpose GPUs
 - Heterogeneous CPU/GPU systems
 - CPU for sequential code, GPU for parallel code
- Programming languages/APIs
 - DirectX, OpenGL
 - C for Graphics (Cg), High Level Shader Language (HLSL)
 - Compute Unified Device Architecture (CUDA)

Concluding Remarks

- Goal: higher performance by using multiple processors
- Difficulties
 - Developing parallel SW
 - Devising appropriate architectures
- SaaS importance is growing and clusters are a good match
- Performance per dollar and performance per Joule drive both mobile and WSC